Project Report Prestifilippo Mattia – Out-of-Core MergeSort

1. The Problem

The goal of the project is to design a scalable MergeSort algorithm to sort a file containing **N records**, where each record has a **variable size** and is composed of an 8-byte sorting key called **key**, the payload length in bytes called **len**, whose value is in the range **[8, PAYLOAD_MAX]**, and a variable-length **payload**. The only field used for comparisons is **key**.

```
struct Record {
  unsigned long key;   // sorting key, 8 bytes
  uint32_t len;       // payload length in bytes (8 ≤ len ≤ PAYLOAD_MAX)
  char payload[];
};
```

The size of the file containing the records to be sorted can be larger than the available RAM, so it may not be possible to load the entire file into memory.

The main difficulty is that the records have a variable size. It is not possible to directly access record *i*.

To scan the file, it is necessary to read the key, read the length len, and then read or skip the len bytes of the payload before moving to the next record.

We observe that the sorting is performed on the key field and not on the payload. Therefore, the payload does not need to be considered during sorting, allowing us to avoid moving a large amount of data.

2. General Idea of the Algorithm

All versions of the project follow the same logical structure, divided into two phases.

Phase 1: Creation of Sorted Runs

The input file is read sequentially in blocks called **chunks**. Each chunk contains a certain number of complete records.

For each chunk, the records are copied into a memory buffer.

For each record stored in the buffer, a lightweight index called **RecordIndex** is created. It contains the sorting key "**key**", the payload length "**len**", and the memory **offset** where the record starts inside the buffer. In this way, the sorting process does not move the payloads contained in the records. This choice reduces unnecessary copies and is especially important when the payloads are large.

The **vector** of **RecordIndex entries** is **sorted** by the key field using **std**::**sort**(). The records are then written back to disk following the order defined by the sorted indices, together with their payloads.

The result of the first phase is a sequence of temporary sorted files, called **runs**: run_0.bin, run_1.bin, ..., run_n-1.bin

Each run is internally sorted, but the complete file is not yet globally sorted.

Phase 2: K-way Merge

The second phase merges all the sorted runs into a single final sorted file.

A **K-way merge** is used, which merges **K sorted runs** at the same time.

For each run, only the current record is kept in memory. The current keys are inserted into a **min-heap**.

At each step, the minimum key is extracted from the **K keys** stored in the heap, together with its corresponding record. The extracted record is written to the output file. Then, the next record from the same run is read, and its key is inserted into the heap. This process continues until all the records from all the runs have been consumed.

If the number of runs is greater than **K**, the merge is performed in multiple passes. The runs are divided into groups. Each group is merged into a new intermediate run using the K-way merge.

The intermediate runs are then merged again. The process continues until a single final run is obtained.

3. Common Modules

record.hpp

This module defines the format of the records read from the input file and the functions used to read record headers from the input file, skip the payload, and write records to the output file.
It is used by both versions of the project: **OpenMP** and **FastFlow**.
The data layout is the following: **[key (8 bytes)][len (4 bytes)][payload (len bytes)]**
Therefore, the size of each record is: 12 + len bytes.
A RecordHeader structure is used to store key and len, representing the header of a record.

An inline function called **readHeader()** is used to read the header of a record from an open file and store it into a RecordHeader.
A local buffer of 12 bytes is used to read key (8 bytes) and len (4 bytes) in a single operation. The function reads 12 bytes from the file and stores them in the buffer.
We use **fread_unlocked()** instead of fread() to avoid the overhead of the libc mutex lock/unlock performed at every call. In our case, a **single thread** is responsible for reading the file sequentially, so there are **no concurrent accesses** to the same stream. Therefore, the unlocked version **avoids unnecessary synchronization**.
If no bytes are read, the end of the file has been reached, and the function returns false.
Using memcpy(), the first 8 bytes of the buffer are copied into the key field, and the next 4 bytes are copied into the len field of the output RecordHeader.

The inline function **skipPayload()** is used to skip the payload of a record in the file without reading it into memory. It uses **fseeko(),** which moves the file cursor of file f by len bytes from the current position.

The inline function **writeRecord()** is used to write a complete record (header + payload) to a file. It takes as input the output file, the key, the payload length len, and the payload itself.
It rebuilds the header in a local buffer of 12 bytes, so that: buf[0..7] = key (8 bytes), buf[8..11] = len (4 bytes) using **memcpy**().
It then writes first the header and then the payload to the file. In this case as well, fwrite_unlocked() is used instead of std::fwrite().

chunk_sorter.hpp

This module implements **Phase 1** of the algorithm, where the sorted runs are created.
A large input file is read sequentially in blocks. For each block, called a **chunk**, an OpenMP task is created to sort the chunk in parallel and generate a sorted run.
Only one thread reads the file sequentially, while the worker tasks receive the chunks that have been read and sort them in parallel.

A **structure** called **RecordIndex** is defined to represent the elements that must be sorted, excluding the payload. In the vector to be sorted, we do not store the entire payload because it can be very large. Instead, we store only the sorting key **key**, the record position **offset** inside the buffer, and the payload length **len**. Therefore, the payload is never moved during **std::sort().**
The **< operator** used by std::sort() is also defined. It compares the key field of two RecordIndex elements.

A **structure** called **ChunkData** is also defined. It represents the input data of an OpenMP task associated with a chunk to be processed.
It contains a raw buffer with all the records of the input chunk, a vector of RecordIndex elements to be sorted, and the path of the run file to be created.

A function called **freeChunk()** is provided to release the memory allocated for a ChunkData object. It deallocates both the buffer and the structure itself.

The function **sortRangeToRuns()** represents **Phase 1** of the out-of-core MergeSort. Its purpose is to read the input file in chunks and launch tasks that sort the chunks and create sorted runs.
The function opens the input file for reading.
A parallel region is then created. The main shared variables are the input file, the directory where the runs will be stored, the chunk size, the vector containing the paths of the generated runs, and the window that limits how many tasks can be launched at the same time.
The parallel region starts with the execution of a single thread. This thread reads the file sequentially, creates the chunks, and launches the tasks that perform sorting and writing.
There are two nested loops. The **outer loop** iterates until the end of the file is reached. Each iteration **reads** and creates a **complete chunk** to be sorted, and **launches a task** that sorts it. The **inner loop** reads **one record** at e**ach iteration** and **inserts** it into the **chunk**. It continues until a **complete chunk** has been filled.
In the **outer loop**, a new chunk is created at each iteration. The chunk is represented by the ChunkData structure. An estimate is made of how many records will fit into the chunk, assuming an average payload size of at least 64 bytes. This estimate is used to reduce continuous reallocations. Memory is then allocated to store the estimated number of records.
In the **inner loop**, complete records are read until the chunk becomes full. **Each** iteration **inserts one record** into the **chunk**. The record header is read and stored in a RecordHeader variable using the utility function readHeader(). If the call fails, it means that the end of the file (EOF) has been reached. A check is then performed to see whether the current chunk is full.
If the next record does not fit into the chunk, it cannot be consumed or discarded. Therefore, the file position is moved back to the beginning of the record header, and the current iteration of the inner loop ends. In the next iteration of the outer loop, that record will be read again as the first record and inserted into the next chunk.
Still inside the inner loop, the offset where the record starts inside the buffer is saved, and the header is copied into the chunk buffer. The payload is then read and stored in the chunk buffer, again using fread_unlocked().
A new **RecordIndex** element is created, and the information needed for sorting is stored in it: the key, the payload length len, and the offset where the record starts inside the chunk buffer. The newly created **RecordIndex** is **inserted** into the **chunk index vector**, which will later be sorted.
The inner loop continues until the chunk is full. At that point, the loop terminates.
In the outer loop, once a complete chunk has been created for sorting, the name of the future sorted run file is assigned and added to the vector containing the paths of the sorted runs.
The **parallel part** then **starts**, where a **task** is **launched** to **sort** the chunk. The task has the ChunkData structure, which represents the chunk, as its private data. The function **sortChunkAndWriteRun(),** executed by each task, is used to **sort** the chunk in parallel and **write it** to disk. The OpenMP directive used is: **#pragma omp task firstprivate(chunk) default(none)**

Outside the directive, the counter of launched tasks is incremented.
A check is performed on the **task_window** limit, which restricts the maximum number of tasks that can run at the same time. When the number of running tasks reaches task_window, the main thread waits until all running tasks have completed before continuing.
Without this limit, the main thread could read the file too quickly and create a very large number of chunks in RAM.
At this point, if the end of the file has been reached, the outer while loop terminates. The function then waits for all launched tasks to complete. This is done using: **#pragma omp taskwait**. Finally, the input file is closed.

The function **sortChunkAndWriteRun()** is executed by a worker thread to sort a vector of RecordIndex elements by key using std::sort() and to create a sorted run.
The function sorts the vector of RecordIndex elements by key, using std::sort() and the < operator implemented in RecordIndex. The vector of RecordIndex elements, now sorted by key, is then scanned, and the records are written to the output file in the new order. To write a record, the corresponding RecordIndex element is taken, and the offset of the record inside the buffer is retrieved. The size of the data to be written is then computed as: header size + len.
Starting from the offset, a single fwrite_unlocked() call of size HEADER_SIZE + len is performed. This process is repeated for all the RecordIndex elements in the sorted vector.
To reduce the number of disk accesses, the output stream is buffered using a 4 MB buffer. In this way, the calls to fwrite_unlocked() accumulate data in the stream buffer and transfer them to the operating system in larger blocks.
Once the run has been completely written, the file is closed and the memory associated with the chunk is released.

kway_merger.hpp
This module implements **Phase 2** of the out-of-core MergeSort, where the merge of multiple sorted files (runs) into a single sorted file is performed.
An auxiliary wrapper structure called **RunReader** is used to model the data read from a sorted run.
The structure stores the pointer to the opened run file, the key of the current record, the length of the current payload, the buffer containing the current payload, and a boolean value that indicates whether the run has finished.
RunReader has a **constructo**r that opens the file and preloads the first record. It initializes all the fields and checks whether the file has been opened correctly.
RunReader also has a method called **advance(),** which reads a record and advances to the next record of a run. The method reads the header of the next record and its payload, setting the attributes to the values that have been read.

Another structure called **HeapEntry** is defined to model an **entry** of the heap. It contains the **key** and the **index** of the **run** from which it comes. The payload is not included for performance reasons.
To implement the min-heap, **std::priority_queue** of HeapEntry elements is used. By default, std::priority_queue is a **max-heap**. The **> operator** used by the priority queue is defined in order to obtain a min-heap. By passing std::greater<HeapEntry> as the comparator, the element with the smallest key is always placed at the top.

A function called **moveOrCopyRun()** is used in the trivial cases where there is only one run. It takes as input the names of the source and destination files, and a deleteSrc flag. If the source and destination file names are the same, the function does nothing.

If the deleteSrc flag is enabled, the source file is renamed to the destination file. Otherwise, the source file is copied to the destination file without deleting the source file.

The function **mergePass()** merges a group of **K runs** into a single sorted file.
It takes as input a vector containing the paths of the runs to be merged, the path of the output file, and a boolean value indicating whether the source runs should be deleted.
K is given by the length of the vector containing the paths of the runs to be merged.
If **K** is equal to 1, there is only one already sorted run, so there is nothing to merge. In this case, the function **moveOrCopyRun()** is used.
Otherwise, a vector of RunReader objects is created, and one RunReader is allocated for each run in the group. The output file for the merge pass is then opened.
A min-heap is initialized, using **std::priority_queue** as the data structure and std::greater as the comparator, so that the **top()** method of the priority queue always returns the HeapEntry with the **minimum key**.
The heap is initialized with the first record of each non-empty run. For each of the **K** runs, the first record is read, represented as a HeapEntry, and inserted into the heap.
A loop is then executed until the heap becomes empty. As long as there are records to process, the entry with the **minimum key** is **extracted** and **written** to the **output file**.
Afterwards, the corresponding reader is advanced using advance(), and the new entry is inserted into the heap.
Outside the loop, the output file is closed, the input files are closed, and they are removed if requested.

4. OpenMP Module
The OpenMP module uses chunk_sorter.hpp to read chunks from the input file and launch the tasks that sort them into output runs. The OpenMP logic is integrated into the **sortRangeToRuns()** function and has been explained previously. While the main thread reads the input file in chunks, the tasks launched by it sort the runs.
To merge the runs into a single final run, the module **kway_merger.hpp** is used, in particular the function **mergePass().**

The module **omp_kway_merger.hpp** extends this logic by introducing parallelism at the level of independent groups of runs.
The function **ompKwayMerge()** is an orchestrator of the multi-pass merge.
If, in a merge pass, there are more than **K** groups of runs to be merged, each group can be assigned to a different OpenMP task.
For example, with merge_fan = 64, if there are 50 input runs, one output run is produced. If there are 200 input runs, 4 intermediate runs are processed in parallel, producing 1 final output run in the next pass.
There is a **K-ary tree** of runs, where the leaves are the input runs, the intermediate nodes are the intermediate runs, and the root is the final output run.
The input parameters are the vector containing the paths of the runs to be merged, the path of the final output file, the merge fan, that is, how many runs are merged in a single mergePass(), deleteRuns, which indicates whether the input files are removed after each pass, and the parallel_merge flag, which indicates whether independent groups of runs are processed as parallel OpenMP tasks.

Inside the function, there is a vector that contains the runs that still have to be merged. It is initialized with the input vector, and at each pass it is replaced by the runs produced by the previous level.

There is a loop that iterates until the number of runs to be merged is reduced to a single final run. Each iteration performs one level of merge passes, executed in parallel. The number of independent groups formed by **K** runs is computed for the current level.
A vector containing the paths of the intermediate output files produced by the current level is initialized.
Since the groups are independent, they can be merged in parallel. A check is performed to verify that the number of groups is greater than 1 and that merge parallelization is enabled. Otherwise, the sequential version is used; if the condition is satisfied, the parallel version is used.
In the parallel case, a parallel region is created, where one task is created for each group of runs to be merged in parallel.
The shared variables are the vector containing the runs to be merged at the current level, the names of the produced runs, the number of groups of runs merged in parallel, and the merge fan **K**.
The following part of the code starts with a single thread. To achieve this, the directives
#pragma omp parallel default(none) shared(sharedVariables)
followed by **#pragma omp single** are used.

There is a **loop** that iterates over each group of runs to be merged. For each group, the start and end indices of the group are computed. A subvector containing the paths of the group is created, containing the elements of the current-level path vector from the computed start index to the computed end index.
A task is launched for each group. Inside the task, the function **mergePass()** is called, passing as parameters the paths of the group of runs and the path of the intermediate output file. To do this, the directive #pragma omp task firstprivate(privateVariables) shared(sharedVariables) is used.
At the end of the outer loop iteration, after performing the merge passes for the entire level and before moving to the next level, there is an implicit barrier that waits for all tasks to complete.
After that, the vector of intermediate output files created at this level is moved into the vector of runs to be merged in the next pass. The algorithm then proceeds to the next level.
Outside the outer loop, once only one final run remains, if current_level still contains one element and it is not already output_path, it is renamed to the final output_path.

omp_sort.cpp
This is the main module that calls the chunk_sorter and the kway_merger of the OpenMP version.
The chunk size in MB, the number of OpenMP threads, the path of the directory where temporary files will be stored, the maximum merge fan **K** for the merge, and the flags used to enable parallelism for groups of runs at the same merge level and to keep the runs are taken from argv.
If they are not provided through argv, they are initialized with default values.
The function sort_to_runs() is executed. Starting from the input file, it produces one sorted run for each chunk.
The function ompKwayMerge() is then called. It performs the multi-level K-way merge, starting from a set of sorted runs and returning a single sorted output file.

ff_chunk_sorter.hpp
Phase 1, in which the file is read in chunks and a sorted run is created for each chunk, is implemented as a **FastFlow farm**, instead of using OpenMP tasks.
There is an **Emitter** that reads the input file, builds the chunks, and sends them to the workers.

There is a set of parallel **Workers**. Each Worker receives a chunk, calls the same sorting function (sortChunkAndWriteRun()) used by the OpenMP version, and writes a sorted run to disk.
There is no collector, because each worker directly produces its own sorted run.

A structure is defined to model the multi-output **Emitter** node of the farm. This node reads a chunk from the input file, represents it as a ChunkData, and sends the ChunkData to the workers.
It has as attributes the input file, the temporary directory used to store the runs, the chunk size, the vector containing the paths of the created runs, and an atomic flag used to signal errors.
The constructor of the Emitter initializes the members with the passed parameters.

The **svc()** method of the Emitter, which represents its logic, starts with an **outer loop** in which the producer continues reading chunks **until** the **end** of the **file** is reached. Each iteration represents the reading of one chunk.
There is an **inner loop** that iterates until the chunk is filled. Each iteration represents the reading of one record. Inside it, the header of the current record is read, the payload of the record is read, and both are inserted into the buffer that models the chunk. Then a RecordIndex is created, storing the key, the len, and the offset in the chunk buffer. The index is added to the index vector.
Once the chunk is full, in the outer loop, the path of the run file is generated and added to the vector of runs. Finally, the chunk is sent to a worker using **ff_send_out(chunk).**
The execution then proceeds to the next iteration, in which the next chunk is read.
If the end of the file has been reached, the outer loop terminates.
If the file reading phase has completed correctly, the workers are notified that no more chunks will arrive by sending an **EOS**.

A structure is then defined to model the **Worker** nodes, of type ff_node_t. The worker receives as input a pointer to a ChunkData.
The **svc()** function calls the sortChunkAndWriteRun() function from chunk_sorter.hpp to sort the chunk and write the run to disk. It returns **GO_ON**, through which the worker communicates that it has finished the task and can receive another one.

The function **ffSortToRuns(),** using the structures created previously, starts and manages the farm to perform the generation of the runs.
It takes as input the path of the input file, the temporary directory that must contain the runs, the chunk size, and the number of workers. It returns a vector containing the paths of the generated runs.
The input file is opened, the emitter is created, a vector of workers is created, and a vector of pointers to workers is created.
There is a loop that executes nWorkers times. At each iteration, a worker is created, added to the worker vector, and its pointer is added to the vector of worker pointers.
Then, the farm is created. The emitter is added to the farm, the vector of worker pointers is added to the farm, and the collector is removed from the farm.
The farm is executed, remaining in passive wait until all tasks have finished.
Once the farm has completed, the input file is closed and the vector containing the paths of the created runs is returned.

ff_kway_merger.hpp
This module implements **Phase 2** of the out-of-core MergeSort. It performs the merge of multiple sorted runs into a single sorted file.

For the K-way merge, the function **mergePass()** from the kway_merger module is used.
The function ompKwayMerge() is reimplemented in a FastFlow version, using exactly the same logic explained previously.

The difference between the OpenMP version and the FastFlow version is that, in OpenMP, one task is created for each group of runs to be merged in parallel.
In this version, instead, the loop that iterates over the groups of runs to be merged is parallelized using ff::ParallelFor, and for each iteration the common function mergePass() is called.
ParallelFor distributes the iterations g = [0, num_groups) among the nWorkers threads using work-stealing.
Each iteration corresponds to an independent group.
The call is blocking, that is, it returns only when all the groups of the current pass have been merged. This acts as a barrier between one pass and the next.
Apart from this, the logic is identical.

The common version used by OpenMP is not used for the following reason.
Phase 1 uses a FastFlow farm, and FastFlow can pin its threads to specific cores through CPU affinity.
If OpenMP were used for the merge immediately after the farm, the OpenMP threads could inherit the same affinity already set by FastFlow.
The groups of runs to be merged at the same level are independent, but they could end up being processed by different OpenMP threads potentially bound to the same core, not fully exploiting parallelism and creating contention.
To avoid this problem, the parallel merge is implemented using ff::ParallelFor.
When the ParallelFor object is instantiated, besides passing the number of workers to use, thread pinning is disabled, avoiding conflicts with the pinning already set by the FastFlow farm of Phase 1.
In this way, both phases are managed by the same FastFlow runtime, avoiding CPU affinity conflicts.
Different groups of runs can therefore be executed in parallel.

ff_sort.cpp
This module is similar to omp_sort.cpp. In the same way, it takes the arguments from argv as input and calls the functions ffSortToRuns() and ffKwayMerge().

6. MPI + OMP Module
mpi_sort.cpp
This module implements the orchestrator of the distributed version of the project.
It does not directly implement the sorting of the records of a chunk or the K-way merge, but coordinates the common modules chunk_sorter and kway_merger across multiple MPI processes.

There is a function called **computeRecordBoundaries()** that divides the input file into numProcs parts of similar size, making sure that each division falls exactly at the beginning of a record.
If we divided the file mathematically (totalSize / numProcs), we could cut a record in the middle, corrupting the sorting process.
The file is scanned sequentially by reading the header and skipping the payload.
When the current offset exceeds the theoretical partition size, the function stops and saves that offset as the division point for the next rank.

There is a function called **mpiSendFile()** that **sends** an **entire file** from the current rank to a destination rank through MPI.
It performs the following optimization.
By dividing the file into blocks of size PIPE_CHUNK, while block *i* is being sent asynchronously through MPI_Isend, the CPU simultaneously reads block *i+1* from disk. This allows the overlap of I/O time with network transfer time, increasing efficiency.
The function sends the size of the file to be transferred to the receiver using MPI_Send.
The file is opened and two buffers are created to read and send the data, in order to implement pipelining.
The first block is read from disk before entering the following loop, filling the send buffer.
A loop is entered in which, as long as there is data to send from the file, the sending of data and the reading from disk continue.
The bytes contained in the send buffer are sent asynchronously to the specified destination.
While the transfer of the current block proceeds in the background, if there are still bytes to read from disk, the next block is read from disk and stored in the read buffer.
The function then waits in a blocking way until the previous asynchronous send has completely finished.
The roles of the two buffers are then swapped: the read buffer that has just been filled becomes the send buffer for the next iteration, and the send buffer that has just been transmitted is used as the read buffer.
When the loop finishes, after all the bytes of the file have been read and sent, the file is closed.

There is a function called **mpiRecvFile()** that receives a file sent by mpiSendFile() and saves it locally on disk. It is symmetric to the sending function. It also uses double-buffer pipelining.
The function receives from the sender the total size of the file to be received. It opens the file in binary write mode in order to store the incoming data. It creates the two alternating receive and write buffers.
It asynchronously receives the first block through **MPI_Irecv().** It then enters a loop that iterates as long as there is data to receive. It waits in a blocking way through **MPI_Wait()** until the reception of the current block has completed. It then swaps the roles of the buffers: the full receive buffer becomes the write buffer, to be written to disk. The write buffer is freed and becomes the receive buffer.
If there are still bytes to receive, the asynchronous reception of the next block is immediately started through MPI_Irecv(), which will be written in the next iteration, before writing to disk the data received previously. While the network receives the new block in the background, the CPU writes the previous block to disk. The loop continues as long as there is data to receive. When the loop finishes, the file is closed.

The main() function is the MPI orchestrator of the distributed program. It is executed in parallel by all the MPI processes that have been started. This code runs on all distributed nodes.
MPI is initialized by specifying that the process uses threads, but only the main thread performs MPI calls.
The id of the current process and the total number of processes started are obtained through MPI_Comm_rank() and MPI_Comm_size().
The parameters and OpenMP are initialized in a way similar to the other two versions.
An **MPI barrier** is used to ensure that all processes have configured their parameters and prepared their resources before proceeding.

Phase 1 then starts. Each process is responsible for a portion of the file, called a **stripe**.

Rank 0 reads the input file serially to correctly compute the file offsets from which each rank starts and ends its processing, using **computeRecordBoundaries().**
Rank 0 broadcasts the boundaries to all ranks, so that every rank knows its exact portion of the file.
Each rank obtains the start and end byte offsets of its exclusive part of the file.
Each rank calls the function sortRangeToRuns(), which reads its portion of the file in chunks, creates sorted runs, and writes them to disk using OpenMP.
It then performs the multi-pass K-way merge by calling the function kwayMerge(), reducing the runs to a single sorted run.
There is then a barrier where all ranks wait until every rank has its local sorted run ready before starting the data exchange between ranks.

Phase 2 then starts. The goal is to merge all the sorted files until a single final sorted file is obtained, which will be owned by rank 0.
The merge is performed as a tree. At each level, a rank sends its run to a neighboring receiver. Each sender, after sending its file, effectively exits the merge process.
The receiver performs a 2-way merge between its own run and the received run, and then continues to the next level. This continues across multiple levels until rank 0 obtains a single sorted run.
In the code, this logic is implemented with a loop in which the variable step represents the distance between a receiver and its corresponding sender.
At each level of the tree, the size of the group of ranks being merged also doubles.
If step = 1, then: groupSize = 2. The groups are: [0,1] [2,3] [4,5] [6,7]
If step = 2, then: groupSize = 4. The groups are: [0,1,2,3] [4,5,6,7]
If step = 4, then: groupSize = 8. The group is: [0,1,2,3,4,5,6,7]
Inside each group, the receiver is the rank located at the beginning of the group.
The sender is the rank located at a distance of step from the beginning of the group.
The ranks that, at a given level, are neither senders nor receivers do not need to do anything and are simply skipped. In this way, the work is distributed across multiple levels, reducing the bottleneck on the master.

If the node executing the code is a **sender** node, it sends the entire resulting run obtained by it to its target receiver, using the function mpiSendFile().
Having completed its work, the sender deletes its intermediate runs and stops participating in the merge process.
If the node executing the code is a **receiver** node, it receives the file sent by the sender through mpiRecvFile(). It then executes a **2-way kwayMerge()** to merge its own sorted file with the received one. When the loop finishes, rank 0 owns the final sorted run.
After that, every node calls MPI_Finalize().

7. Performance Evaluation

OpenMP Single Node

The benchmark was executed on a single node using the OpenMP implementation with the following parameters: **Records:** 50,000,000. **PAYLOAD_MAX:** 4096 B. **Chunk size:** 64 MB. **Merge fan-in:** 8. **Number of threads:** 1–32.
The chunk size and merge fan values were chosen experimentally by comparing the execution time and speedup of different parameter combinations.
Smaller chunk sizes lead to the generation of more runs. Smaller merge fan values lead to the generation of more groups. With these parameters, the sort phase generates 36 runs. The runs are merged in groups of 8.
The following results were obtained:

Threads used	Sort time (s)	Merge time (s)	Total time (s)	Sort Speedup	Merge Speedup	Total Speedup	Efficiency
1	14.57	20.20	34.77	1.00×	1.00×	1.00×	100.0%
2	12.12	18.97	31.09	1.20×	1.06×	1.12×	55.9%
4	6.80	15.03	21.83	2.14×	1.34×	1.59×	39.8%
8	5.05	14.26	19.31	2.88×	1.42×	1.80×	22.5%
16	4.66	13.43	18.10	3.12×	1.50×	1.92×	12.0%
32	4.23	13.51	17.74	3.44×	1.50×	1.96×	6.1%

The **phase** that **generates** the **initial runs** is the one that benefits the **most** from **parallelism**. The increase from 1 to 4 threads produces the largest gain, reducing the sorting time by more than half.
Beyond 8 threads, the benefits become progressively smaller, since the speedup of the **sort phase**, and consequently of the total execution, starts to flatten.
The **bottleneck** is the **merge phase**, in which the generated runs are read, merged, and written in parallel groups of 8 runs.
This phase scales **poorly**, with a maximum internal speedup of only 1.5×, which flattens around 4 threads, where the merge time stabilizes at about 13–15 seconds.
This is due to the fact that merge is an intrinsically **I/O-bound** operation.
Overall, increasing the number of threads reduces the execution time, but with progressively lower efficiency.

Single Node FastFlow
The measurements on the FastFlow version were performed using the same parameters as the previous measurement.
Results

Threads used	Sort time (s)	Merge time (s)	Total time (s)	Sort Speedup	Merge Speedup	Total Speedup	Efficiency
1	14.39	20.99	35.38	1.00×	1.00×	1.00×	100.0%
2	11.54	20.19	31.73	1.25×	1.04×	1.12×	55.8%
4	6.32	16.56	22.88	2.28×	1.27×	1.55×	38.7%
8	5.40	13.80	19.21	2.66×	1.52×	1.84×	23.0%
16	4.67	13.45	18.12	3.08×	1.56×	1.95×	12.2%
32	4.47	14.22	18.69	3.22×	1.48×	1.89×	5.9%

The observations are similar to those of the previous experiment, both for the sort phase and for the merge phase. Up to 8 threads, compared to the OpenMP version, FastFlow shows a slightly better sort time, while the total execution time is always worse. The bottleneck is still represented by the merge phase. The efficiency scales in the same way as in the OpenMP version.

Payload Distribution Analysis
This experiment evaluates the performance of the OpenMP implementation by varying the maximum payload size and the number of records while keeping the total data volume constant. Two datasets were used: 8 million records, maximum payload size 512 B. 2 million records, maximum payload size 2048 B.
Analysis
As the payload size increases and the number of records decreases, the sorting algorithm performs fewer operations because the payload is ignored during comparisons. As a result, the sort time decreases.

The speedup and efficiency behavior remains similar in both datasets. It can be observed that the merge speedup stabilizes around 1.50× beyond 8 threads in both cases, confirming that the merge phase remains the main bottleneck.

Dataset: mediumPayload8M (8M records, 512 B payload)

Threads used	Sort time (s)	Merge time (s)	Total time (s)	Sort Speedup	Merge Speedup	Total Speedup	Efficiency
1	4.24	6.30	10.54	1.00×	1.00×	1.00×	100.0%
8	1.89	4.10	5.99	2.25×	1.54×	1.76×	22.0%
32	1.75	4.15	5.90	2.42×	1.52×	1.79×	5.6%

Dataset: largePayload2M (2M records, 2048 B payload)

Threads used	Sort time (s)	Merge time (s)	Total time (s)	Sort Speedup	Merge Speedup	Total Speedup	Efficiency
1	2.97	4.68	7.65	1.00×	1.00×	1.00×	100.0%
8	1.52	3.05	4.57	1.96×	1.53×	1.68×	20.9%
32	1.36	3.06	4.43	2.18×	1.53×	1.73×	5.4%

MPI+OMP Multi-Node Strong Scaling

This benchmark measures the **strong scaling** of the distributed MPI+OMP implementation. The dataset remains fixed at **200 million records** (~9.6 GB), while the number of nodes scales from 1 to 8. For each node configuration, the number of OpenMP threads per rank is also varied.

The MPI speedup is calculated with respect to the absolute baseline of 1 node × 1 thread (179.14 s). For cross-configuration comparisons, the **best single-node** result with 16 threads (116.74 s) is also used, as it represents the practical limit of a single node before saturation.

The following **parameters** were used: **Dataset**: 200 million records. **Topology**: 1 rank per node, nodes = 1 / 2 / 4 / 8. **Threads per rank**: 1 / 4 / 8 / 16 / 32. **Chunk size**: 64 MB. **Merge fan-in**: 8. **Generated runs**: 144 per local rank.

Single Node

Threads/Rank	Phase 1 (Sort)	Phase 2 (Merge)	Total	Speedup vs 1t
1	179.14 s	~1.3×10^{-6} s	179.14 s	1.00×
4	153.38 s	~1.3×10^{-6} s	153.38 s	1.17×
16	116.74 s	~1.3×10^{-6} s	116.74 s	1.53× (Best)
32	117.38 s	~1.3×10^{-6} s	117.38 s	1.53×

On a single node, the duration of phase two is essentially zero, since there is no data to transfer between ranks, while the execution time is concentrated in phase 1. The total execution time stabilizes at 16 threads, while with 32 threads it becomes worse.

Two Nodes

Threads/Rank	Total Cores	Phase 1 (Sort)	Phase 2 (MPI Merge)	Total	Speedup vs 1N×1t	Speedup vs 1N×16t
1	2	97.22 s	35.75 s	**132.97 s**	1.35×	0.88×
4	8	95.49 s	35.85 s	**131.34 s**	1.36×	0.89×
8	16	69.91 s	36.66 s	**106.57 s**	1.68×	1.10×
16	32	67.49 s	36.64 s	**104.13 s**	1.72×	1.12× (Best)

Threads/Rank	Total Cores	Phase 1 (Sort)	Phase 2 (MPI Merge)	Total	Speedup vs 1N×1t	Speedup vs 1N×16t
32	64	74.58 s	36.68 s	**111.26 s**	1.61×	1.05×

The MPI multi-node merge is stable (~35.8 s) and insensitive to the number of threads. The sort time is lower, but the gained time is lost during the distributed merge because of data transfers (I/O) in a tree structure among the different nodes. The best execution time is obtained with 16 threads.

Four Nodes

Threads/Rank	Total Cores	Phase 1 (Sort)	Phase 2 (MPI Merge)	Total	Speedup vs 1N×1t	Speedup vs 1N×16t
1	4	71.52 s	69.62 s	**141.14 s**	1.27×	0.83×
4	16	58.72 s	60.61 s	**119.33 s**	1.50×	0.98×
8	32	56.02 s	69.37 s	**125.39 s**	1.43×	0.93
16	64	55.20 s	55.64 s	**110.84 s**	1.62×	1.05× (Best)
32	128	54.88 s	55.49 s	**110.37 s**	1.62×	1.06×

With 4 nodes, the sort phase improves, but the distributed merge phase **explodes** to **55–69 seconds**, becoming the dominant phase in the total execution time. The total speedup compared to the single-node configuration with 16 threads is only **slightly positive** or even **negative**. Only with 16–32 threads per rank does Phase 1 decrease enough to compensate for the cost of the distributed merge.

Eight Nodes

Threads/Rank	Total Cores	Phase 1 (Sort)	Phase 2 (MPI Merge)	Total	Speedup vs 1N×1t	Speedup vs 1N×16t
1	8	60.66 s	80.42 s	141.08 s	1.27×	0.83×
4	32	53.85 s	80.08 s	133.93 s	1.34×	0.87×
8	64	52.95 s	80.89 s	133.84 s	1.34×	0.87×
16	128	53.41 s	81.06 s	134.47 s	1.33×	0.87×
32	256	52.03 s	80.94 s	132.97 s	1.35× Best	0.88×

With 8 nodes, the MPI distributed merge phase stabilizes at approximately **81 s**, independently of the number of threads, dominating the total execution time and eliminating any benefit from local parallelism.

Observations

Phase 1, the sort phase, scales as the number of nodes and threads increases, because each node works on an **independent stripe** of the dataset. The distributed merge phase grows **almost linearly** with the number of nodes, because as the number of ranks increases, the volume of data that must be transferred and merged also increases.

Weak Scalability

To perform this measurement, the following approach was used. Each node always processes the same amount of data (~0.5 GiB, 8 chunks of 64 MB). The execution time is measured, the amount of data is divided by the execution time, and the result is multiplied by 180, determining how many GiB a node and, overall, the entire system can process within a **fixed budget** of **180 seconds**.

Nodes	Threads/Rank	Phase 1 (Sort)	Phase 2 (MPI Merge)	Total	Total Capacity (GiB/180s)	Capacity per Node (GiB/180s)
1	1	5.96 s	~0 s	5.96 s	15.11	15.11
1	32	3.24 s	~0 s	3.24 s	27.82	27.82
2	1	10.26 s	4.13 s	14.39 s	12.51	6.26
2	16	7.47 s	4.12 s	11.59 s	15.53	7.76
2	32	8.54 s	4.24 s	12.78 s	14.08	7.04
4	1	15.70 s	16.19 s	31.89 s	11.29	2.82
4	16	13.98 s	15.85 s	29.83 s	12.07	3.02
4	32	13.95 s	15.78 s	29.73 s	12.11	3.03
8	1	26.89 s	35.87 s	62.76 s	11.47	1.43
8	8	24.60 s	36.26 s	60.87 s	11.83	1.48
8	16	24.75 s	35.82 s	60.58 s	11.89	1.49
8	32	24.74 s	35.77 s	60.51 s	11.90	1.49

The **total capacity**, expressed as the amount of GiB that can be processed by the entire cluster in 180 seconds, **decreases** as the number of nodes increases instead of growing. The system **processes less data** using 8 nodes than using a single well-configured node.

The more nodes are added, the more the overhead of the distributed merge increases, and the more the total capacity decreases. Paradoxically, the single-node version with 32 threads is the one with the highest total capacity.

Each additional node provides computational contribution, but the increasing overhead of the distributed merge cancels and exceeds this benefit. The weak scaling of this algorithm is structurally limited by the distributed merge phase.

8. Cost Model

Model parameters: Let ***numRecords*** be the total number of records, ***avgPayload*** the average payload length per record, ***dimFile*** the total file size (*numRecords · (12 + avgPayload)*), ***chunkSize*** the size of each chunk in RAM, ***runsPerRank*** the number of runs produced per rank (*dimFile / (numRank · chunkSize)*), ***numRank*** the number of MPI ranks, ***numThreads*** the number of OMP threads per rank, ***fanMerge*** the number of runs merged per group in the multilevel merge, ***netCostByte*** the cost per byte transmitted

over the network, ***latencyCost*** the fixed latency per MPI message, ***tRead*** the time required to read 1 byte from disk, and ***tWrite*** the time required to write 1 byte to disk. The **total cost** is composed of the cost of **Phase 1** plus the cost of **Phase 2**.

Phase 1 — Local Sort

Each rank processes a stripe of ***dimFile/numRank*** bytes and produces ***runsPerRank*** = *dimFile/(numRank·chunkSize)* sorted runs.

Stripe I/O cost:

readStirpe = (dimFile/numRank) · tRead

writeStirpe = (dimFile/numRank) · tWrite

In-memory sort: The sort does not move the payloads, but only a vector of *RecordIndex = {key, offset, len}* of about 20 B each. The comparison cost is negligible compared to the I/O cost.

Local multi-pass merge: To perform the local merge, each rank must merge ***runsPerRank*** runs in groups of ***fanMerge***.

Therefore, $\lceil \log_{fanMerge}(runsPerRank) \rceil$ passes are required. Each pass reads and rewrites *dimFile/numRank* bytes:

$$tMerge = \lceil \log_{fanMerge}(runsPerRank) \rceil \cdot (dimFile/numRank) \cdot (tRead + tWrite)$$

Total Phase 1 cost (per rank): The cost consists of the initial stripe read plus the merge cost.

$$tPhase1 = (dimFile/numRank) \cdot tRead \cdot \left(1 + \lceil \log_{fanMerge}(runsPerRank) \rceil\right) + (dimFile/numRank) \cdot tWrite \cdot \lceil \log_{fanMerge}(runsPerRank) \rceil$$

Phase 2 — Binary Tree Merge

At the step with distance s, each sender transmits a file of $(dimFile/numRank) \cdot s$*bytes.*

Communication cost: $tComm(s) = latencyCost + (dimFile/numRank) \cdot s \cdot netCostByte$

2-way merge cost of the receiver: *The receiver merges two files of* $(dimFile/numRank) \cdot s$*bytes:* $tMerge(s) = (dimFile/numRank) \cdot s \cdot (2 \cdot tRead + 2 \cdot tWrite)$

Total communication cost: *Sum over the* $\log_2(numRank)$*steps* $(s = 1,2,4, \dots, numRank/2)$*. The latency cost is paid for each of the* $\log_2(numRank)$*steps. Each step transfers* $(dimFile/numRank) \cdot s \cdot netCostByte$

The steps are $s = 1,2,4, \dots, \frac{numRank}{2}$*, that is, a geometric series whose sum is equal to* $numRank - 1$*. Therefore, the total amount of data transferred along the critical path is* $netCostByte \cdot \frac{dimFile}{numRank} \cdot (numRank - 1)$*, which becomes* $dimFile \cdot \left(1 - \frac{1}{numRank}\right)$*.*

The ***total communication cost*** *becomes:* $\sum T_{comm} = \log_2(numRank) \cdot latencyCost + dimFile \cdot netCostByte \cdot \left(1 - \frac{1}{numRank}\right)$

Total merge cost: Each step $s$has a merge cost of $(dimFile/numRank) \cdot s \cdot (2 \cdot tRead + 2 \cdot tWrite)$ for each receiver.

Each step costs $2 \cdot \frac{dimFile}{numRank} \cdot s \cdot (tRead + tWrite)$. Therefore, the sum becomes $2 \cdot \frac{dimFile}{numRank} \cdot (numRank - 1) \cdot (tRead + tWrite)$, that is, $2 \cdot dimFile \cdot \left(1 - \frac{1}{numRank}\right) \cdot (tRead + tWrite)$.

Thus: $\sum T_{merge} = 2 \cdot dimFile \cdot (tRead + tWrite) \cdot \left(1 - \frac{1}{numRank}\right)$

Total Phase 2 cost: $tPhase2 = \sum T_{comm} + \sum T_{merge} = \log_2(numRank) \cdot latencyCost + dimFile \cdot netCostByte \cdot \left(1 - \frac{1}{numRank}\right) + 2 \cdot dimFile \cdot (tRead + tWrite) \cdot \left(1 - \frac{1}{numRank}\right)$